Lecture 13.
Parameter-Efficient Fine-Tuning

Luneva Natalia

05.05.2025

PEFT Motivation

- High Cost of Full Fine-Tuning
- Limited Data for Fine-Tuning
- Scalability
- Resistance to Catastrophic Forgetting

PEFT Motivation

- **High Cost of Full Fine-Tuning**
  - **Computational Complexity.** Significant computational resources are required to update all parameters
  - **Storage Costs.** Each new task requires storing the entire model, increasing memory requirements
- Limited Data for Fine-Tuning
- Scalability
- Resistance to Catastrophic Forgetting

PEFT Motivation

- High Cost of Full Fine-Tuning
- **Limited Data for Fine-Tuning**
  - Full fine-tuning can lead to overfitting, especially if the model is very large
- Scalability
- Resistance to Catastrophic Forgetting

PEFT Motivation

- High Cost of Full Fine-Tuning
- Limited Data for Fine-Tuning
- **Scalability**
  - Full fine-tuning generates different models, which is inefficient when the same model needs to be adapted for different tasks or domains
- Resistance to Catastrophic Forgetting

PEFT Motivation

- High Cost of Full Fine-Tuning
- Limited Data for Fine-Tuning
- Scalability
- **Resistance to Catastrophic Forgetting**
  - During full fine-tuning, the model may "forget" knowledge acquired during pretraining

PEFT Overview

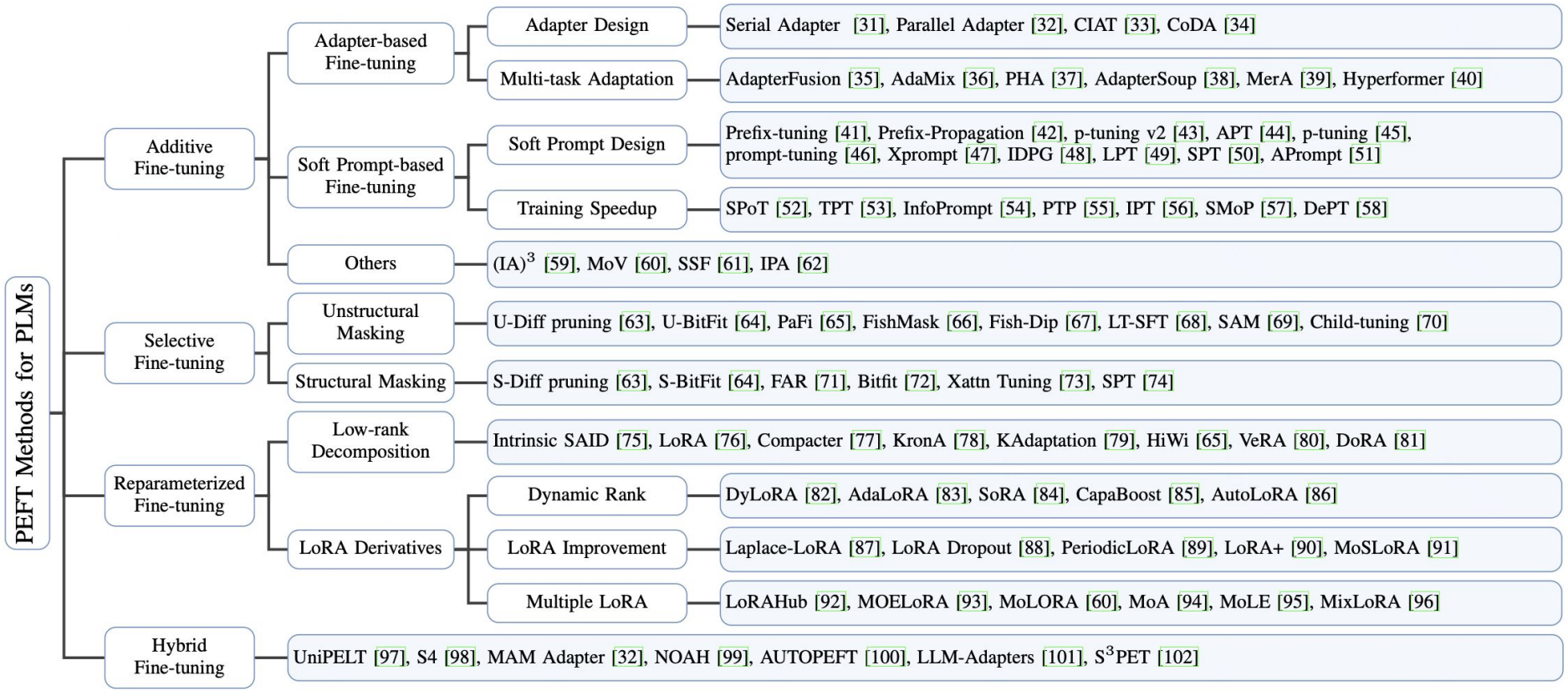
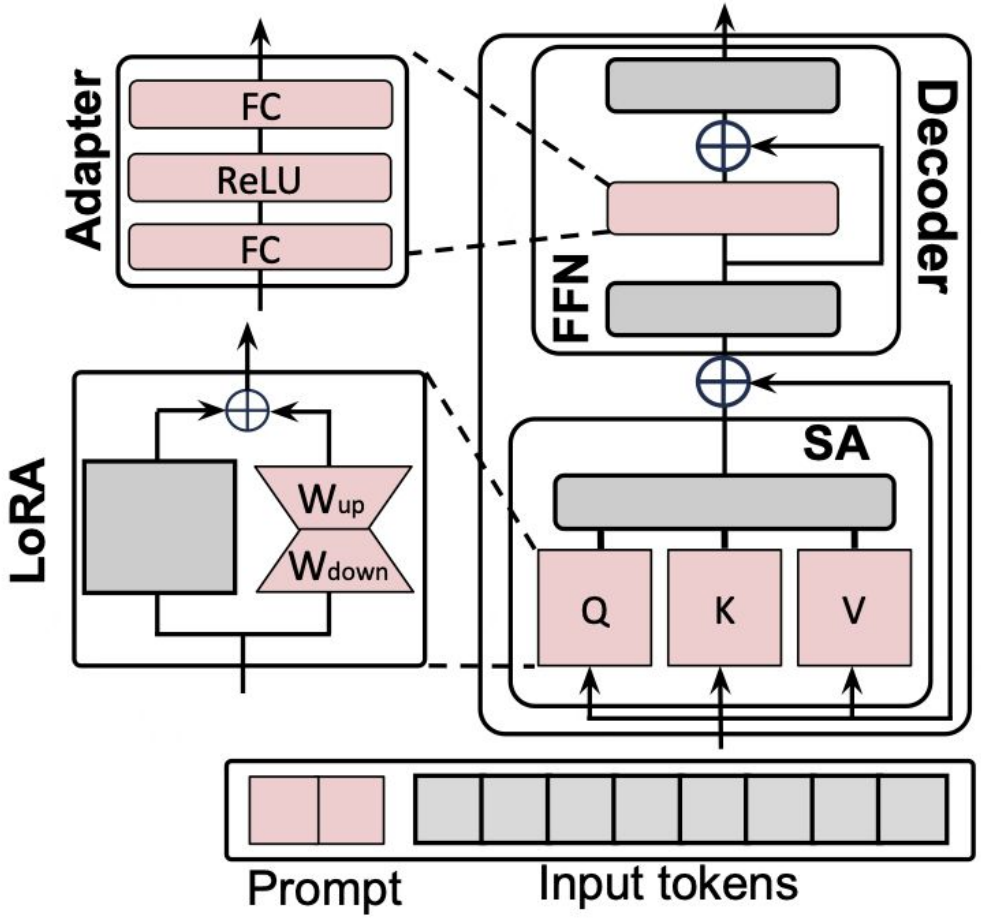


Fig. 3: Taxonomy of Parameter-Efficient Fine-Tuning Methods for Large Models.

2403.14608

PEFT Methods

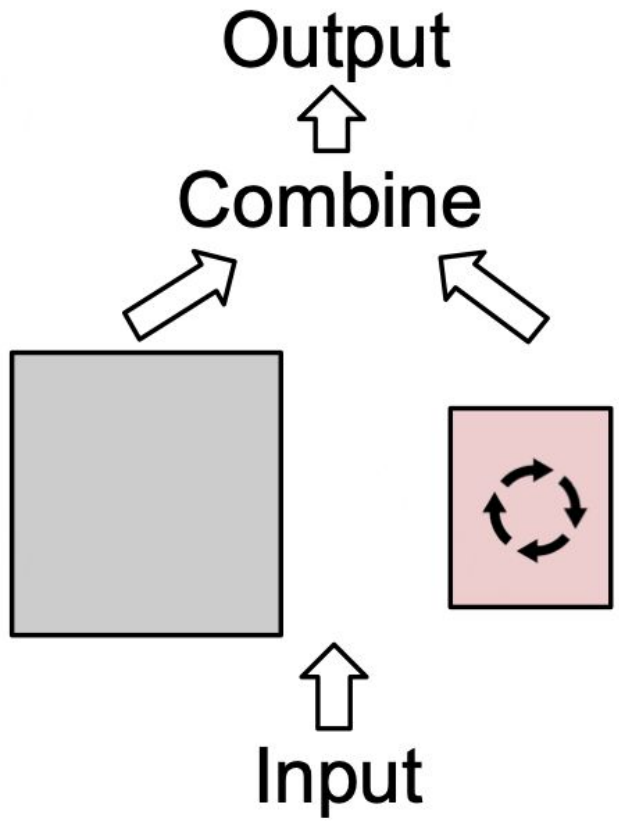
- **Additive**
  - Adapters
  - Soft Prompts
- **Selective**
- **Reparametrization**
  - LoRA family
- **Hybrid**



2403.14608

Additive PEFT

- Introduce only a minimal number of trainable parameters and strategically add them to the model architecture
- During FT, only the weights of these additional modules are updated

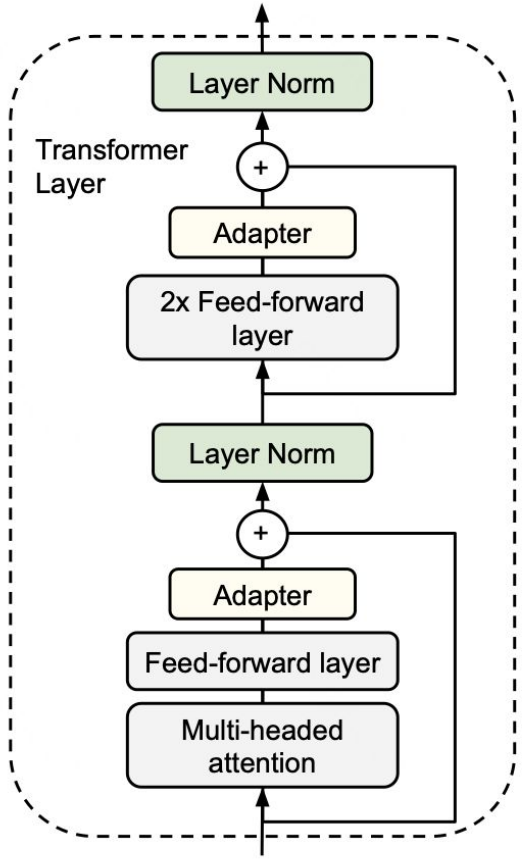


2403.14608

Adapter Layers

- Consists of two linear transformations (down- and up-projections) with a non-linearity in between
- The dimensionality of the down-projection is significantly **smaller** than the original space

$$\text{Adapter}(x) = W_{up}\sigma(W_{down}x) + x$$



1902.00751

Prompting

- **Hard Prompts** consist of manually created *natural text* that instructs the model about the task
- **Soft Prompts** utilize continuous and *trainable vectors* that are concatenated to input embeddings

$$X^{(l)} = [s_1^l, \ldots, s_{N_s}^l, x_1^l, \ldots, x_{N_x}^l]$$

where $X^{(l)}$ is the sequence of input tokens for layer l, including soft prompt tokens

Soft Prompts

- **Prompt-Tuning** and **P-Tuning** apply learnable vectors only at the initial word embedding layer

Prompt-tuning is effective only in the context of the large models (more than 11B parameters)

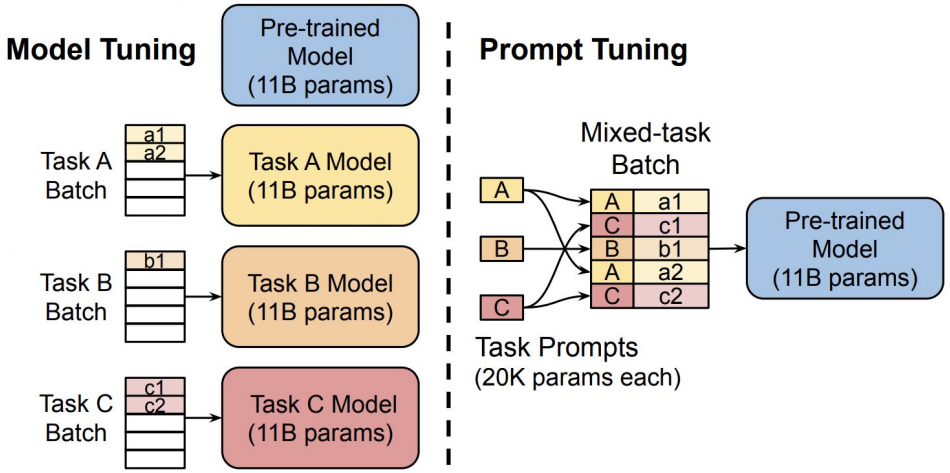


Figure 2: **Model tuning** requires making a task-specific copy of the entire pre-trained model for each downstream task and inference must be performed in separate batches. **Prompt tuning** only requires storing a small task-specific prompt for each task, and enables mixed-task inference using the original pre-trained model.

2104.08691

Soft Prompts

- **Prefix-Tuning** introduces learnable vectors that are added at the beginning of the keys *k* and values *v* across **all** Transformer layers

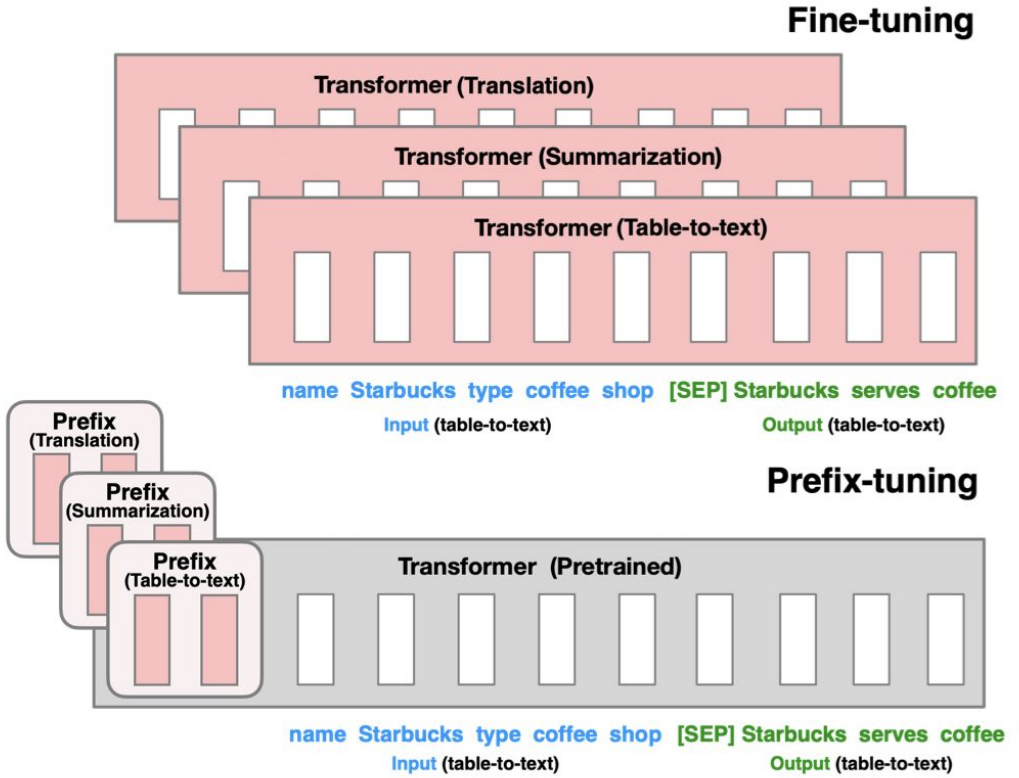


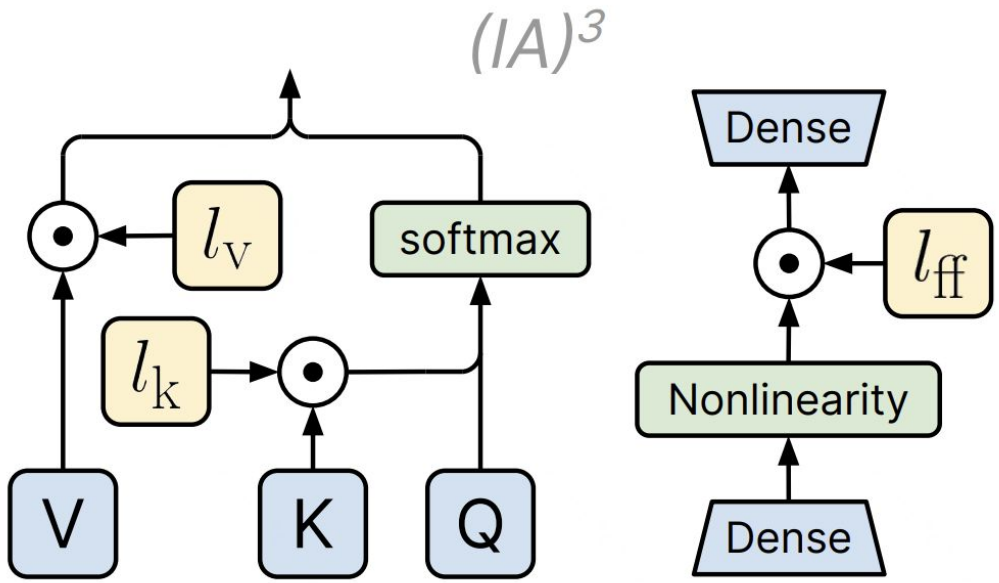
Figure 1: Fine-tuning (top) updates all Transformer parameters (the red Transformer box)

2101.00190

$(IA)^3$

Let's leave this for self-study or RG)))

- TI;dr. $(IA)^3$ modifies only specific learned vectors associated with **key**, **value**, and **feedforward** layers within transformer blocks
- It introduces three vectors, l_v, l_k, and l_f, which rescale (via element-wise multiplication) activations in attention and ff layers

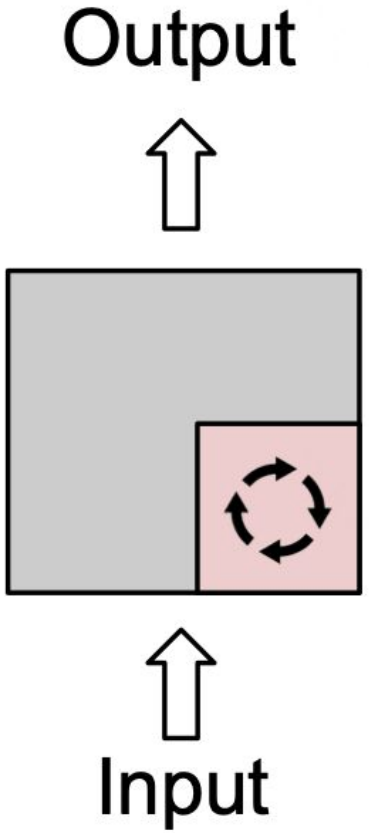


2205.05638

Selective PEFT

- Fine-tunes a subset of the existing parameters

$$\theta = \{\theta_1, \theta_2, \ldots, \theta_n\}$$
$$M = \{m_1, m_2, \ldots, m_n\}, m \in \{0, 1\}$$
$$\hat{\theta}_i = \theta_i - \eta \cdot m_i \cdot \frac{\partial L}{\partial \theta_i}$$



2403.14608

Selective PEFT

Examples:

- **Top Layer Fine-Tuning.** Focusing on fine-tuning only the upper layers of the network (e.g. classification head) while leaving the lower layers untouched
- **Specific Parameter Fine-Tuning.** Selectively training certain types of parameters (e.g. biases) while freezing other parameters

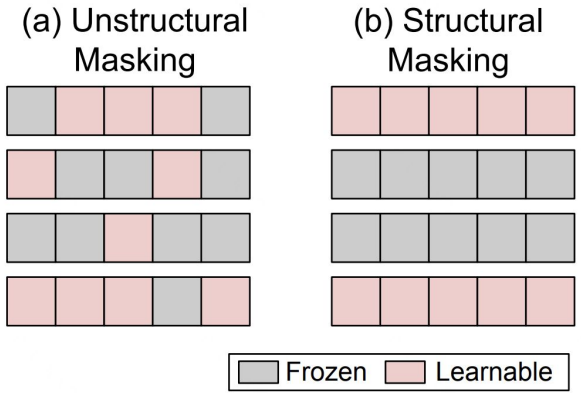
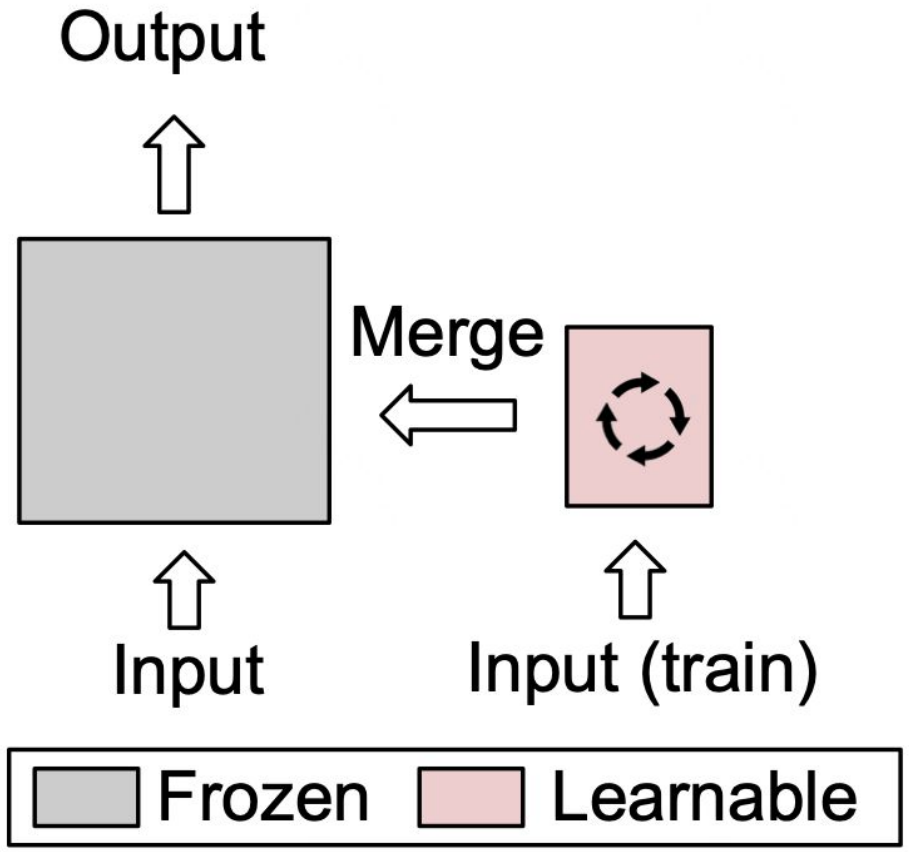


Fig. 7: Illustration of two parameter masking methods.

2403.14608

Reparameterization PEFT

- Stands for **equivalently** transforming a model's architecture from one to another via transforming its parameters



2403.14608

Low-Rank Adaptation

- Pretrained over-parametrized models have a low intrinsic dimension
- Hypothesis: change in weights during model adaptation also has a low “intrinsic rank”

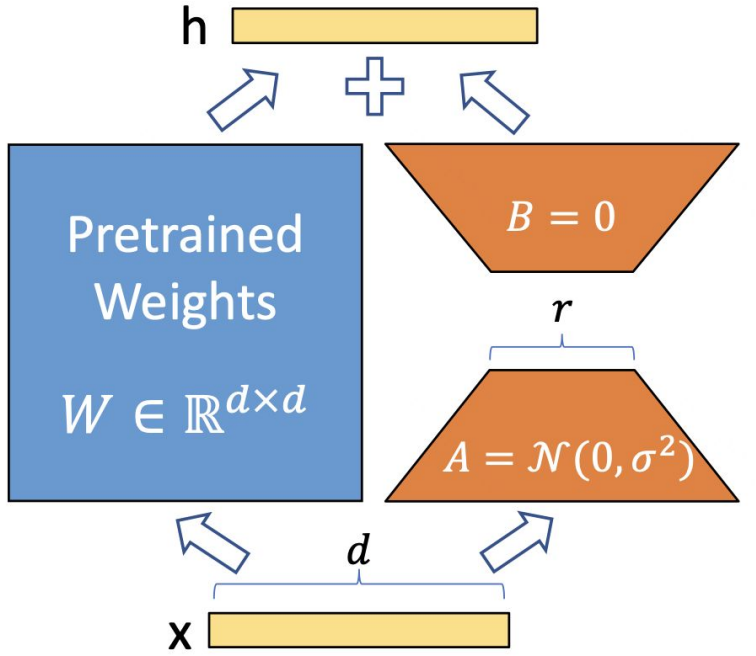


Figure 1: Our reparametrization. We only train A and B.

2106.09685

Low-Rank Adaptation

- $W \in R^{d \times k}$ — pretrained weight matrix
- $B \in R^{d \times r}$, $A \in \mathbb{R}^{r \times k}$
- $B = 0$, $A \in N(0, \sigma^2)$
- $r \ll min(d, k)$ — rank (hyperparameter)
- $\mathrm{h} = W\mathrm{x} + \frac{\alpha}{r}\Delta W\mathrm{x} = W_0\mathrm{x} + \frac{\alpha}{r}BA\mathrm{x}$
- $\frac{\alpha}{r}$ — scaling factor
  - Divide by r for robustness
  - Multiply by α for impact control

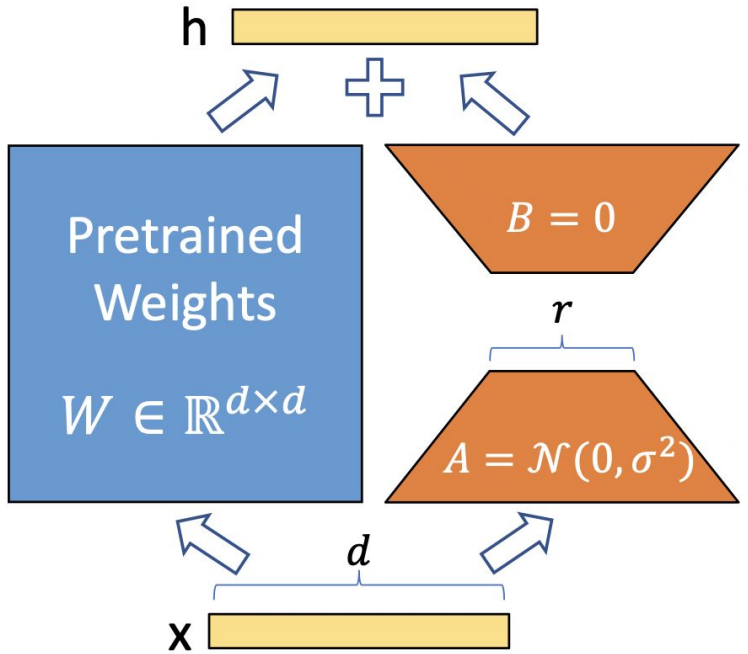


Figure 1: Our reparametrization. We only train A and B.

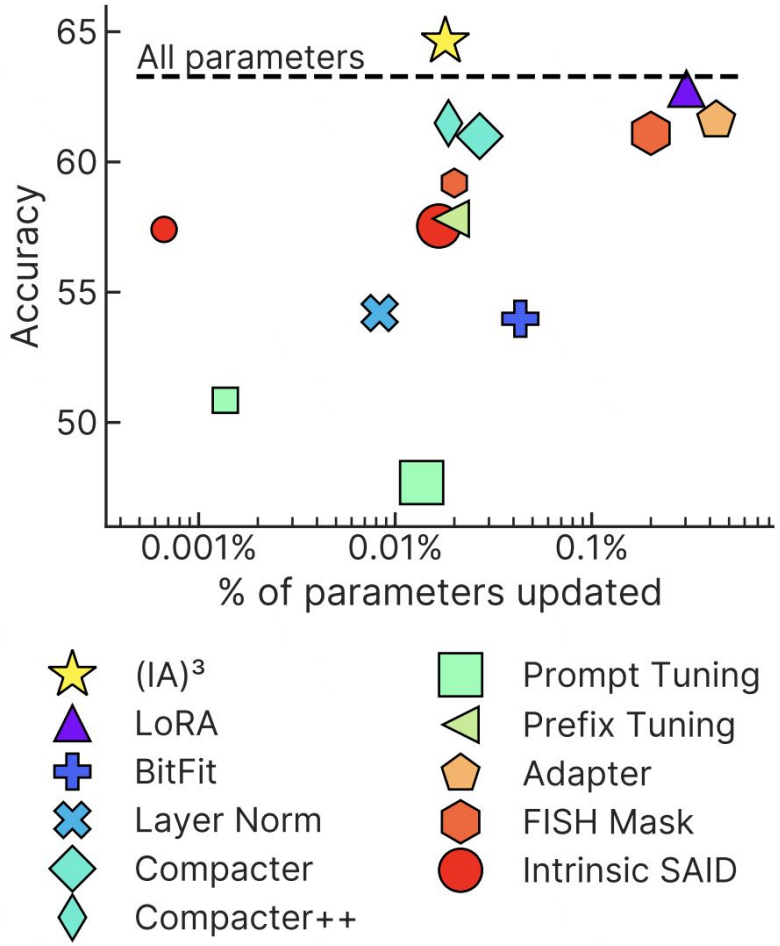
2106.09685

What to choose?

Method	Type	Efficiency Disk	RAM	BP	Inference overhead
Adapters (Houlsby et al., 2019)	A	✓	✓	✗	+ FFN
AdaMix (Wang et al., 2022)	A	✓	✓	✗	+ FFN
SparseAdapter (He et al., 2022b)	AS	✓	✓	✗	+ FFN
Cross-Attn tuning (Gheini et al., 2021)	S	✓	✓	✗	No overhead
BitFit (Ben-Zaken et al., 2021)	S	✓	✓	✗	No overhead
DiffPruning (Guo et al., 2020)	S	✓	✗	✗	No overhead
Fish-Mask (Sung et al., 2021)	S	✓	✗[5]	✗	No overhead
LT-SFT (Ansell et al., 2022)	S	✓	✗[5]	✗	No overhead
Prompt Tuning (Lester et al., 2021)	A	✓	✓	✗	+ input
Prefix-Tuning (Li and Liang, 2021)	A	✓	✓	✗	+ input
Spot (Vu et al., 2021)	A	✓	✓	✗	+ input
IPT (Qin et al., 2021)	A	✓	✓	✗	+ FFN and input
MAM Adapter (He et al., 2022a)	A	✓	✓	✗	+ FFN and input
Parallel Adapter (He et al., 2022a)	A	✓	✓	✗	+ FFN
Intrinsinc SAID (Aghajanyan et al., 2020)	R	✓	✗	✗	No overhead
LoRa (Hu et al., 2022)	R	✓	✓	✗	No overhead
DoRA (Liu et al., 2024)	R	✓	✓	✗	No overhead
UniPELT (Mao et al., 2021)	AR	✓	✓	✗	+ FFN and input
Compacter (Karimi Mahabadi et al., 2021)	AR	✓	✓	✗	+ FFN
PHM Adapter (Karimi Mahabadi et al., 2021)	AR	✓	✓	✗	+ FFN
KronA (Edalati et al., 2022)	R	✓	✓	✗	No overhead
KronA^{B}_{res} (Edalati et al., 2022)	AR	✓	✓	✗	+ linear layer
$(\text{IA})^3$ (Liu et al., 2022)	A	✓	✓	✗	+ gating
Attention Fusion (Cao et al., 2022)	A	✓	✓	✓	+ decoder
LeTS (Fu et al., 2021)	A	✓	✓	✓	+ FFN
Ladder Side-Tuning (Sung et al., 2022)	A	✓	✓	✓	+ decoder
FAR (Vucetic et al., 2022)	S	✓	✓	✗	No overhead
S4-model (Chen et al., 2023)	ARS	✓	✓	✗	+ FFN and input

Table 1: Comparing PEFT methods across storage (**disk**), memory (**RAM**), and computational efficiency in terms of reducing backpropagation costs (**BP**) and **inference overhead**. The ✓ means that the method is more effective than full fine-tuning, the ✗ means that it is less efficient than full fine-tuning.
Types: A – additive, S – selective, R – reparametrization-based.

2303.15647



2205.05638

What to choose?

		Additive			Selective		Reparam.	Hybrid
		Adapters	Soft prompts	Other	Structured	Sparse		
Storage efficiency		✓	✓	✓	✓	✓	✓	✓
Training	RAM	✓	✓	✓	✓	✗	✓	maybe
	Speed	✗	✗	maybe	maybe	✗	✓	maybe
Inference	Single-task	✗ Adds overhead			✓ No overhead		✓	maybe
	Mutli-task	✓ Allows efficient inference			maybe	✗	maybe	maybe
Implemented in		🤗	🤗	🤗	✗	✗	🤗	🤗

2303.15647

PEFT Overview

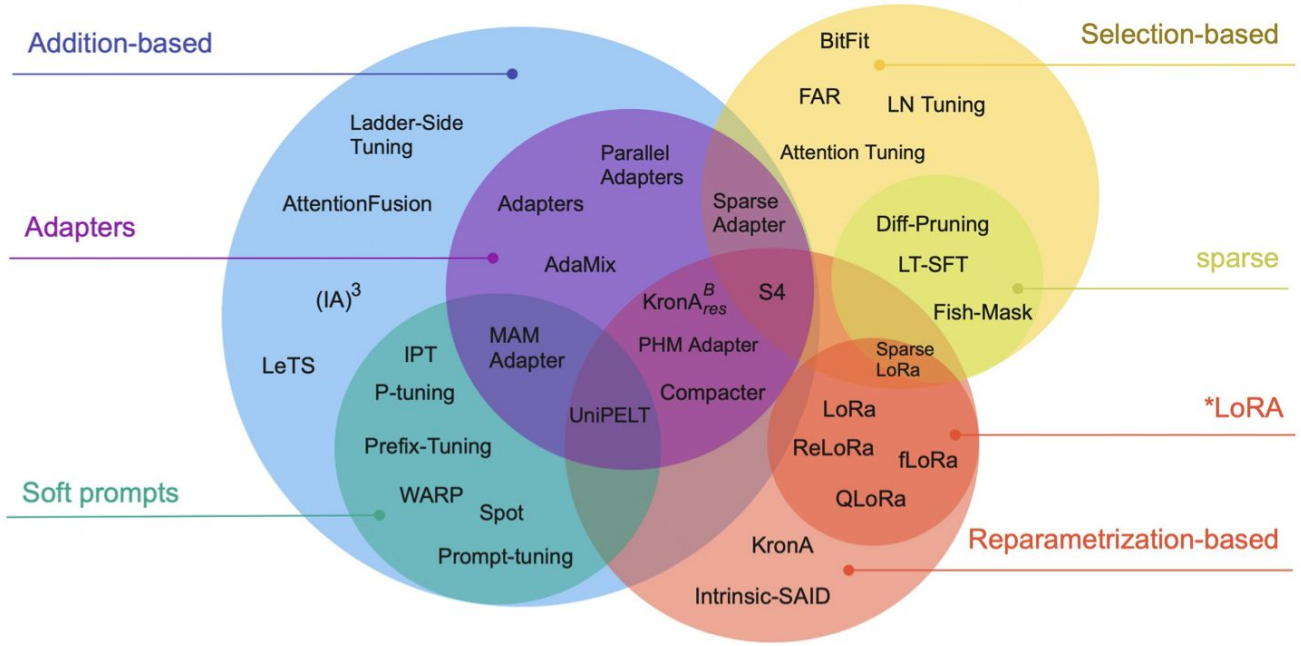


Figure 2: Parameter-efficient fine-tuning methods taxonomy. We identify three main classes of methods: **Addition**-based, **Selection**-based, and **Reparametrization**-based. Within additive methods, we distinguish two large included groups: **Adapter-like** methods and **Soft prompts**.

2303.15647

References

Overview:

- https://arxiv.org/pdf/2403.14608
- https://arxiv.org/pdf/2303.15647

LoRA

- https://arxiv.org/pdf/2106.09685

Prompt-tuning

- https://arxiv.org/pdf/2104.08691

P-tuning

- https://arxiv.org/pdf/2103.10385

Prefix-tuning

- https://arxiv.org/pdf/2101.00190

Self-Study Materials

(IA)3

- https://arxiv.org/pdf/2205.05638

LoRA family

- https://huggingface.co/docs/peft/conceptual_guides/adapter
- https://arxiv.org/pdf/2305.14314

Adapters

- https://arxiv.org/pdf/1902.00751

FishMask

- https://arxiv.org/pdf/2111.09839

PEFT overall

- https://github.com/huggingface/peft
- https://huggingface.co/blog/samuellimabraz/peft-methods